

Paramount+ — Client Technical Interview Prep

Role: Sr. AWS Python Developer — Media Platform

Client: Paramount+ (via LTIMindtree / Okaya)

Interview: Fri Aug 7, 2026 · **6:00 AM PST** · Virtual

Comp confirmed: \$140K + benefits (below \$155K target — negotiate if advanced)

Resume: `applications/paramount/resume.pdf`

Due Aug 6: LTM Self-Identification form on RippleHire (login cgjonesdev@gmail.com)

[Gmail — pre-offer doc request](#) · [Gmail — pre-interview doc](#) · [Varun Verma \(LTM\)](#).

Interview thread: [Gmail — Okaya scheduling](#) · [Anurag Chaudhary \(Okaya\)](#).

What this round is

LTM recruiter screen is **done** (Varun Verma, Jul 31). This is the **Paramount+ client technical** — expect Python + AWS depth, media/platform scenarios, and possibly live coding or system design.

Round	Focus	Duration
1 — Recruiter / HM	LTM → Paramount+ submission	Done ✓
2 — Technical	Python + AWS (Lambda, ECS, event-driven)	45–60 min
3 — System design	Media ingest, catalog, partner APIs	45–60 min (may combine)
4 — Team / final	Culture, onsite, comp	30 min (later)

Pre-flight checklist

- ☐ Complete RippleHire Self-ID form by **Aug 6**
 - ☐ Join virtual interview **Fri 6:00 AM PST** — test audio/video night before
 - ☐ Resume PDF open: `applications/paramount/resume.pdf`
 - ☐ Quiet room, water, notepad for names and next steps
 - ☐ Block 60 min on calendar (6 AM is early — sleep plan matters)
-

60-second intro

Bullet cues (glance, don't read):

- **Who:** Cary Jones · Senior Software Engineer · **15+ years**
- **Python/AWS:** Production serverless + containers — Lambda, ECS/Fargate, event-driven pipelines
- **Disney:** Media & entertainment backend — \$10M+ daily, batch 8hr→45min, 99.99% uptime
- **CGJSoftware:** Lambda/ECS microservices, 2.5M daily transactions, <120ms latency
- **Why Paramount+:** Build scalable media platform systems — ingest, catalog, high-availability APIs
- **AI tools:** Cursor/Copilot in dev workflow — tests, refactors, never unreviewed ship

"I'm Cary Jones, a Senior Software Engineer with 15 years building Python backends on AWS. At Disney I worked on media and financial reporting backends at enterprise scale — \$10 million plus daily, 99.99% uptime — including batch pipelines I cut from eight hours to forty-five minutes. Through CGJSoftware I've architected event-driven Python services on Lambda and ECS processing millions of daily transactions. I'm excited about Paramount+ because this role is exactly that intersection: scalable media platform engineering, serverless and container patterns, and production rigor. I use AI coding tools daily to accelerate tests and refactors, but I always review and own what ships."

Behavioral — likely questions

Q1. Tell me about yourself / why Paramount+?

Probe: Disney media, AWS scale, W2 motivation.

15 yrs Python/AWS. Disney = media & entertainment backend, \$10M+ daily metrics, 99.99% uptime. CGJSoftware = Lambda/ECS event-driven pipelines, 2.5M daily transactions. Want to build scalable media platform systems at Paramount+ on a long-term product team.

Polish: Say *batch processing, quadrants, disbursement*. CGJSoftware = **2.5M daily transactions** (volume). Add **99.99% uptime** at Disney; one W2 line if probed.

If follow-up: "How does Disney map to streaming?" → Enterprise media/financial backends; same SLAs (volume, pipeline reliability). Not consumer playback, but production rigor matches platform work.

Q2. Disney experience — relation to streaming/media platforms?

At Disney I was on enterprise media and financial reporting — high-throughput pipelines, strict uptime, cross-team integration during platform changes. When Disney acquired Hulu, our group was part of the streaming ecosystem: higher release cadence, media-adjacent dependencies,

consumer platform pressure. I owned backend batch and reporting on AWS; the media org context Paramount+ operates in is familiar.

Scope: Own backend batch/reporting on AWS; cite Hulu acquisition exposure and streaming-adjacent environment. Frame as enterprise media pipelines + platform migration pressure.

Polish: Drop one metric — batch **8hr → 45min** or **99.99% uptime**.

Q3. CGJSoftware — what did you build? Why W2 now?

Consulting entity for B2B/C2C contracts — Python microservices on Lambda/ECS, event-driven messaging, CI/CD ownership. Not a gap; active end-to-end delivery. Moving to W2 for long-term product team, shared codebase, and Paramount-scale media platform impact.

Add if light on detail: ~2.5M daily transactions, sub-120ms on hot path.

W2 why (expand): Shared roadmap, code review culture, multi-year product ownership vs. project-by-project consulting.

Q4. How do you use AI coding tools in production?

Cursor/Copilot/Claude for boilerplate, tests, refactors — coverage 65%→92%, delivery +35%. Never ship unreviewed AI code; enforce pytest, ruff, human review. Aligns with JD requirement.

Optional close: "AI accelerates drafts; I still own design, edge cases, and production behavior."

Q5. Improved reliability or performance under load (STAR)

Pick one:

- **Disney (payment processing):** Monolith SQL functions for month-end payroll — DBAs avoided touching it. Pitched API + lightweight UI; parallelized hub-and-spoke ingest (Jira, Airtable, costing). **8hr → 45min** after 3 months.
- **CGJSoftware:** ECS latency <120ms, +45% throughput — connection pooling, cache layer, auto-scaling

Polish: Say *quadrants, disbursement*. Tie to Paramount: "parallel ingest from multiple sources, idempotent stages, observable batch jobs."

STAR shape: Situation = monolith + disbursement office pain → Task = managers wanted API/UI → Action = you modeled backend + parallelized hub-and-spoke → Result = 8hr→45min in 3 months.

Python + AWS technical

Lambda vs ECS Fargate?

- **Lambda:** short-lived, spiky, event-triggered (S3, API Gateway, SQS); low ops; pay per invoke; OK with cold starts
- **Fargate:** long-running APIs, WebSockets, sustained traffic, custom deps, cold-start intolerance
- **Hybrid:** API on Fargate, async workers on Lambda

Polish: Fargate = *long-running / always-on* (APIs, WebSockets). Lambda = cost-efficient for sporadic, event-driven work. Hybrid: API on Fargate, async workers on Lambda.

Media example: Upload webhook → Lambda metadata extract; catalog API on Fargate; transcoding fan-out via SQS → Lambda workers.

Testable Lambda handler structure

Layers: Thin handler (parse event) → service layer (business logic) → repository (DynamoDB/S3). DI for unit tests. Idempotency key from `message_id` / event ID. JSON structured logging. Config from env / Secrets Manager.

Focus: Thin handler → service → repository. Pipeline (SQS/S3/DynamoDB) is separate from code structure inside the function.

One-liner: "Handler is 10 lines — parse, delegate, return. All logic lives in a service class I can pytest without AWS."

Idempotency: Check dedupe table keyed on SQS `messageId` before processing; safe on at-least-once delivery.

Event-driven media asset ingestion

S3 upload → EventBridge/S3 notification → SQS → Lambda (validate, metadata) → DynamoDB catalog → Step Functions transcoding → SNS completion. DLQ for failures. At-least-once → idempotent handlers.

30-second interview answer: "Upload goes to S3, the event lands on SQS, Lambda validates and writes metadata to DynamoDB, Step Functions handle transcoding, SNS signals completion, and handlers are idempotent because SQS can deliver again."

Polish: Handlers are idempotent because *SQS can deliver again*. Add DLQ for repeated failures; EventBridge routes the S3 event.

Disney tie-in: Same hub-and-spoke idea — parallel stages, multiple sources, observable batch jobs.

DynamoDB content catalog access patterns

PK = `CONTENT#<id>`, SK = `METADATA | EPISODE#<n>`. GSI1: slug lookup. GSI2: genre#date browse. Design keys around reads; avoid scans.

30-second answer: Single-table design — PK `CONTENT#id`, SK `METADATA` or `EPISODE#n`. GSI on slug for URL lookups; GSI on genre with date-prefixed sort key for browse. Every path is a query/index lookup — no scans.

Polish: Say *keyed lookups, no table scans* for every access path.

Postgres pivot: “Deepest prod DB work is PostgreSQL; I understand PK/SK/GSI and idempotent writes — ramp quickly on catalog patterns.”

Step Functions partial failures

Retry + backoff on transient errors; Catch → DLQ/SNS on permanent failure. Compensation where needed. CloudWatch on state transitions. Idempotent downstream steps for safe replay.

30-second answer: “Retry transient failures with backoff inside the state machine. Catch permanent failures → SNS alert + mark catalog `failed` + optional DLQ/ops replay. Downstream steps idempotent so replays are safe. CloudWatch on state transitions.” Step Functions uses **Retry** and **Catch** on each state.

Example: MediaConvert timeout → Retry 3x → Catch → update DynamoDB status `transcode_failed` → SNS to ops.

System design prompts

Design: process uploaded video files for streaming

Core flow: S3 ingest → EventBridge → SQS → Lambda (validate + metadata) → DynamoDB catalog → Step Functions → MediaConvert transcode → publish to CDN origin → status API.

30-second skeleton: “Upload to S3, queue the event, Lambda validates and writes metadata, Step Functions orchestrate transcoding, outputs land on CDN-backed storage, catalog/API reflects status.”

Include: MediaConvert (or worker fleet) for transcoding — packaged HLS/bitrate outputs, then CDN origin.

Duplicate upload (idempotency): Check catalog for existing `asset_id` + status ≥ `transcoding`; conditional write on first insert; return success without re-enqueueing transcode. Dedupe key = content ID or SQS `messageId`.

One-liner: “If `asset_id` already exists and transcode started, short-circuit — idempotent handler returns success, no double MediaConvert job.”

Design: rate-limited partner API (1000 req/min global, 10 req/sec per partner)

API Gateway usage plans OR Redis token bucket per partner key. Async heavy work via SQS. 429 + Retry-After. CloudWatch per partner.

Coding practice (live or take-home)

Run drills: `python interview/sessions/paramount_coding_drill.py`

#	Problem	JD tie-in
1	LRU Cache	Metadata / session cache
2	Merge overlapping intervals	Broadcast windows / schedules
3	Top K frequent titles	Popularity / analytics
4	Idempotent Lambda handler sketch	Serverless best practice
5	Sliding window rate limiter	Partner API protection
6	Parse HLS #EXTINF manifest	Media metadata

Interview tip: Talk through approach 2–3 min before coding. Target 20–25 min per problem.

Problem 2 — merge intervals (pseudocode)

```
function merge(intervals):
    if intervals is empty: return []

    sort intervals by start
    merged = [copy of intervals[0]]
    for each [start, end] in intervals[1:]:
        last = merged[-1]
        if start <= last.end:           # overlap or touch
            last.end = max(last.end, end)
        else:
            merged.append([start, end])
    return merged
```

$O(n \log n)$ sort + $O(n)$ scan. Edge cases: empty, single interval, one nested inside another.

Problem 3 — top K frequent (common mistakes)

```
def top_k_frequent(ids, k):
    counts = Counter(ids)           # 1. count frequencies
    # 2. pick top k by freq (heap, sort, or bucket)
    return [id for id, _ in counts.most_common(k)]
```

Key: `k` = number of results to return. Use `Counter.most_common(k)` or sort by frequency descending, take first `k` IDs.

Problem 1 — LRU cache (common mistakes)

LRU: evict *least recently used* — track **recency order** in OrderedDict (`move_to_end` , `popitem(last=False)`).

```
from collections import OrderedDict

class LRUCache:
    def __init__(self, capacity):
        self.cap = capacity
        self.cache = OrderedDict()

    def get(self, key):
        if key not in self.cache:
            return None
        self.cache.move_to_end(key)    # mark recently used
        return self.cache[key]

    def put(self, key, value):
        if key in self.cache:
            self.cache.move_to_end(key)
        self.cache[key] = value
        if len(self.cache) > self.cap:
            self.cache.popitem(last=False)    # evict oldest (LRU)
```

Trace cap=2: put(a,1), put(b,2), get(a), put(c,3) → evicts **b** (not used since a was touched).

Problem 5 — sliding window rate limiter

```
class SlidingWindowRateLimiter:
    def __init__(self, limit, window_sec):
        self.limit = limit
        self.window_sec = window_sec
        self._hits = defaultdict(list)    # partner_id → timestamps

    def allow(self, partner_id, now=None):
        now = now or time.time()
        cutoff = now - self.window_sec
        hits = [t for t in self._hits[partner_id] if t > cutoff]
        if len(hits) >= self.limit:
            self._hits[partner_id] = hits
            return False
        hits.append(now)
        self._hits[partner_id] = hits
        return True
```

Say: Per partner, prune hits older than window; if count $\geq$ limit return `False`, else append timestamp and return `True`.

Resume story map

Topic	Story	Metric
Media / enterprise scale	Disney financial & media backend	\$10M+ daily, 99.99% uptime
Batch / pipeline perf	Disney PostgreSQL optimization	8 hr $\rightarrow$ 45 min
Serverless / ECS	CGJSoftware Lambda + Fargate	2.5M daily txns, <120ms
Event-driven	SQS/SNS messaging pipelines	+45% throughput
Testing / CI	pytest, ruff, GitHub Actions	65% $\rightarrow$ 92% coverage
AI-assisted dev	Cursor/Copilot in workflow	+35% delivery speed

Gaps — honest pivots

Gap	Pivot
DynamoDB depth	Strong PostgreSQL + MongoDB; understand PK/SK, GSI, idempotent writes
CDK	CloudFormation/Terraform awareness; eager to adopt CDK for IaC
Step Functions	Designed async workflows with queues; Step Functions = visual orchestration
Media containers (MP4/HLS)	Disney media domain; ingest $\rightarrow$ metadata $\rightarrow$ transcode conceptually
C#	Python-primary; can read .NET interop if needed
K8s	ECS/Fargate production exp; K8s concepts familiar, ready to deepen

Questions to ask Paramount+ interviewers

1. What does the media platform team own — ingest, catalog, playback APIs, or internal tooling?
2. Lambda-heavy vs ECS/K8s split on the team?
3. How is DynamoDB used in the content catalog?
4. CI/CD and testing expectations for serverless deploys?
5. On-site LA vs NY — hybrid flexibility?

6. What does success look like in the first 90 days?

Self-scoring (mock practice)

Score	Meaning
3	Structured answer, metrics, tradeoffs
2	Correct but rambling or missing depth
1	Vague or wrong — review model answer

Interactive mock: `run interview prep paramount` · Full guide:
`interview/sessions/paramount_mock.md`